Actividad. Identificar una aplicación tipo. Documentar la ejecución del SDLC

Datos de los Estudiantes

Nombre: Ruben

Apellidos: Dougall van Kommer

Identificador: 22458337@live.uem.es

Detalles del Curso

Nombre de la Asignatura: Metodologías de desarrollo seguro

Nombre del Profesor: Sergio Iglesias Perez

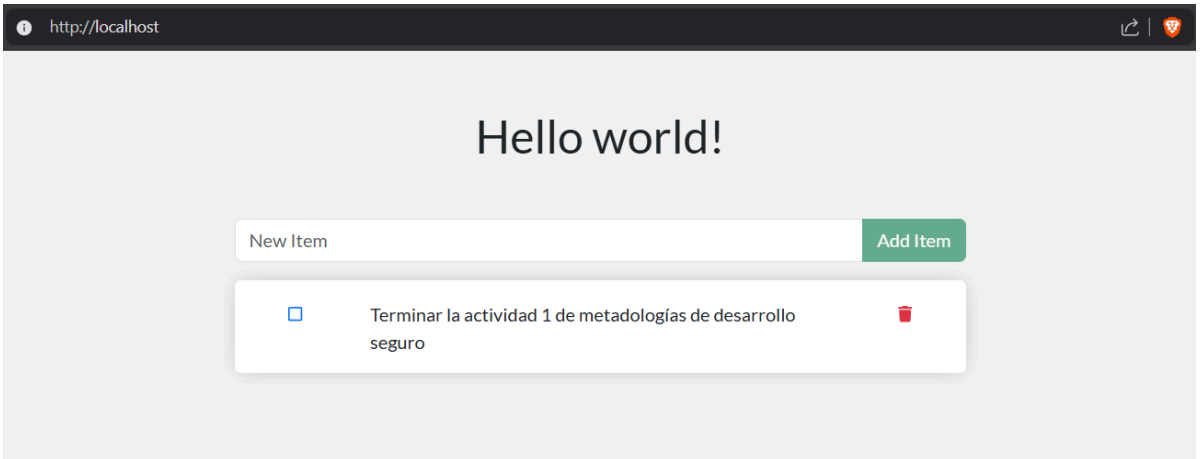
1. Descripción de la aplicación

Para esta actividad he seleccionado la aplicación To-Do.

a. Qué hace la aplicación

La aplicación permite gestionar una lista de tareas y debido a que está ejecutada en Docker, permite disponer de un entorno aislado, reproducible y fácilmente desplegable. Esto sobre todo es muy útil en el desarrollo seguro del software, ya que permite controlar mejor el entorno de ejecución y aplicar configuraciones seguras.
Esta aplicación tiene distintas funcionalidades, las cuales son las siguientes:

- Añadir nuevas tareas
- Visualizar las tareas almacenadas en la base de datos.
- Interactuar con la aplicación desde el navegador.



b. Tipo de aplicación

Es una aplicación web, entonces a la hora de ajustar los parámetros de seguridad tenemos que tener varias cosas en cuenta: inyecciones SQL, establecer mínimo privilegio en en cuentas de la base de datos, evitar XSS (validar la entrada), usar protocolos seguros HTTPS/TLS y no usar protocolos que tienen vulnerabilidades, tener un control sobre el backend y el frontend de la aplicación, …

2. Ejecución de la aplicación

Para ejecutar la aplicación, he seguido los pasos indicados en el tutorial:

1. Descargar o clonar el repositorio del proyecto desde PowerShell:

```
PS C:\WINDOWS\system32> git clone https://github.com/docker/getting-started-todo-app
Cloning into 'getting-started-todo-app'...
remote: Enumerating objects: 217, done.
remote: Counting objects: 100% (101/101), done.
remote: Compressing objects: 100% (65/65), done.
remote: Total 217 (delta 64), reused 36 (delta 36), pack-reused 116 (from 2)
Receiving objects: 100% (217/217), 4.28 MiB | 11.88 MiB/s, done.
Resolving deltas: 100% (104/104), done.
```

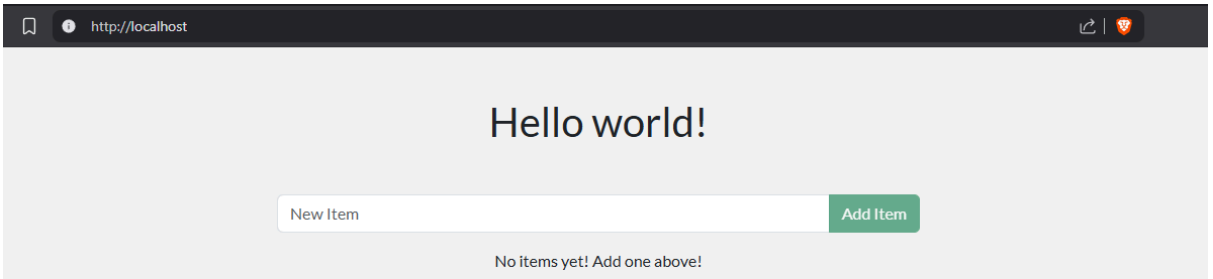
2. Acceder al directorio del proyecto:

```
PS C:\WINDOWS\system32> cd getting-started-todo-app
```

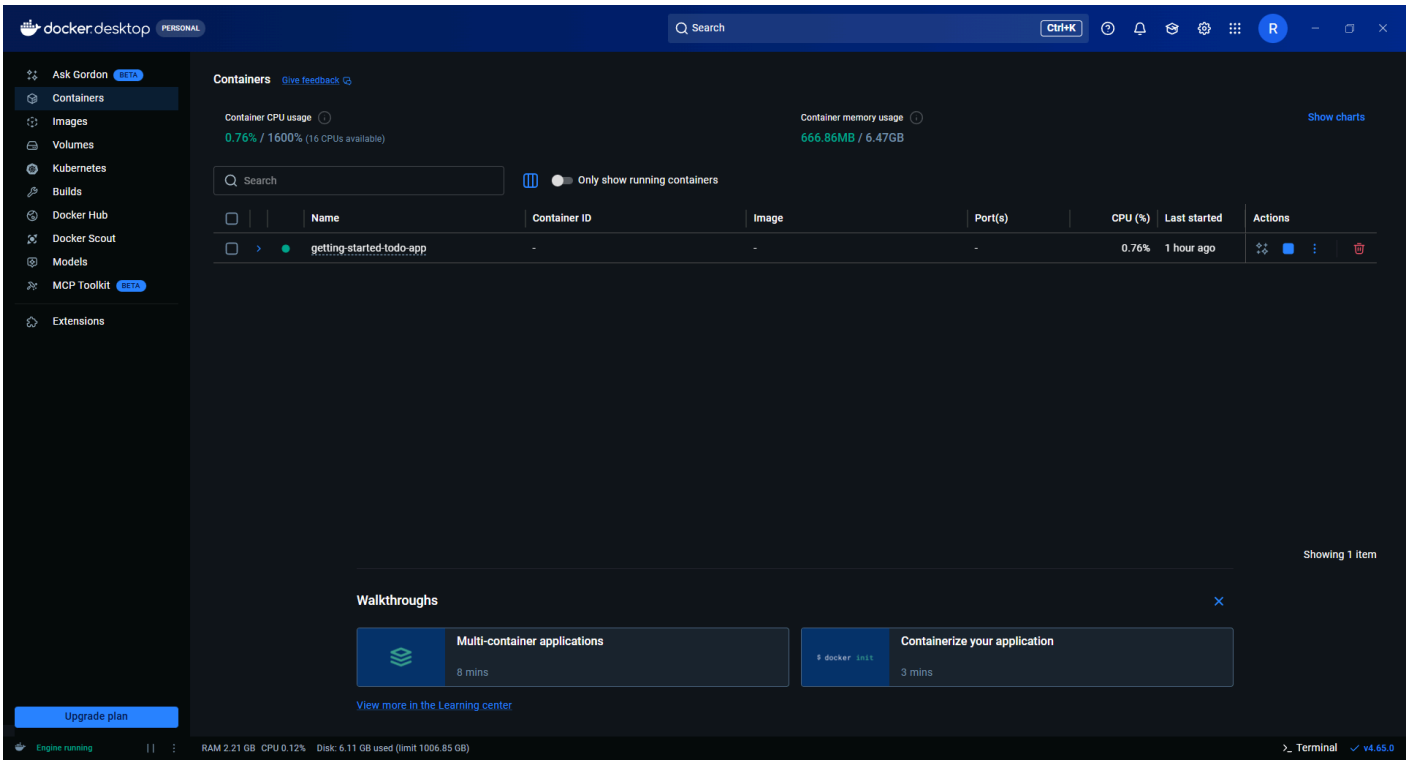
2. Iniciar la aplicación mediante Docker Compose:

```
PS C:\WINDOWS\system32\getting-started-todo-app> docker compose watch
[+] up 43/43
 ✔Image traefik:v3.6 Pulled
 ✔Image phpmyadmin   Pulled
 ✔Image mysql:9.3    Pulled
#1 [internal] load local bake definitions
#1 reading from stdin 1.14kB done
#1 DONE 0.0s

#2 [client internal] load build definition from Dockerfile
#2 transferring dockerfile: 3.22kB 0.0s done
#2 DONE 0.1s

#3 [backend internal] load metadata for docker.io/library/node:22
#3 ...

#4 [auth] library/node:pull token for registry-1.docker.io
#4 DONE 0.0s

#3 [backend internal] load metadata for docker.io/library/node:22
#3 DONE 1.5s
```

3. Una vez iniciados los contenedores, acceder desde el navegador a:



4. Docker: aplicación web To-Do



5. La aplicación se ha ejecutado desde la ruta local mediante Docker Compose.

```
PS C:\Users\ruben> cd "C:\Users\ruben\Downloads\ASIGNATURAS\METODOLOGÍAS DE DESARROLLO SEGURO\ACTIVIDADES\1\getting-started-todo-app"
PS C:\Users\ruben\Downloads\ASIGNATURAS\METODOLOGÍAS DE DESARROLLO SEGURO\ACTIVIDADES\1\getting-started-todo-app> pwd

Path
----
C:\Users\ruben\Downloads\ASIGNATURAS\METODOLOGÍAS DE DESARROLLO SEGURO\ACTIVIDADES\1\getting-started-todo-app
```

3. Requisitos de seguridad

Aquí vamos a definir una serie de requisitos de seguridad que van a garantizar que nuestra aplicación sea segura para ello se implementa desde el inicio en la fase de recopilación de requisitos.

- **Requisitos de seguridad de la aplicación web:**
  - Proteger la confidencialidad de los datos, evitando accesos no autorizados a la información almacenada en nuestro sistema de base de datos.
  - Garantizar la integridad de los datos, para que ninguna persona que no esté autorizada pueda realizar modificaciones.
  - Mantener la disponibilidad del servicio. Esto sobre todo es importante en el caso de que tengamos una aplicación que tenga muchos usuarios conectados al mismo tiempo. Para ello, tiene que ser escalable y flexible al mismo tiempo para evitar interrupciones que impidan el uso normal de la aplicación. También es importante tener en cuenta la posibilidad de recibir ataques DDoS.Para estos ataques es muy recomendable usar servicios de mitigación (Cloudflare, Akamai, …) e implementar cortafuegos de aplicaciones web (WAF).
  - Definir requisitos de autenticación y autorización, para controlar quien accede a la aplicación y qué acciones puede realizar cada usuario (gestión de permisos).

- **Amenazas a considerar:**
  - Acceso no autorizado a la aplicación o a la base de datos.
  - Manipulación de datos introducidos por el usuario o incluso almacenados en el sistema.
  - Ataques de inyección de tipo SQL. Esto sucede cuando una aplicación web no valida la entrada del usuario antes de incluirla en una consulta a la base de datos.
  - Ataques Cross-Site-Scripting, que inyectan scripts maliciosos (generalmente JavaScript) en sitios web legítimos. Cuando un usuario visita el sitio afectado, su navegador ejecuta el código, permitiendo a los atacantes robar cookies de sesión, secuestrar cuentas, redirigir usuarios o alterar la apariencia de la web
  - DDoS, como comentaba antes, podría interrumpir el acceso a la aplicación.
  - Dependencias vulnerables, en el caso de que tengamos alguna librería usada por la aplicación tuviera fallos conocidos.
  - Exposición de servicios o puertos innecesarios en el entorno Docker o en la red. Para ello, se puede usar la herramienta Nmap para realizar un análisis extenso de qué puertos están en

uso y cuales no están realizando ninguna función. También se puede ver qué versiones están corriendo en cada puerto y comprobar si esas versiones son antiguas y pueden tener vulnerabilidades.

- **Riesgos más importantes (en el caso de que esta sea una aplicación web desarrollada en equipo en un contexto realista) son:**
    - La filtración de datos es muy importante, ya que es un incidente de seguridad en el que información confidencial, como datos personales, financieros o secretos comerciales, es robada. Esto conlleva graves consecuencias, como fraudes, robo de identidad y sanciones económicas. Esto puede deberse a ataques de hackers, errores humanos, malas configuraciones o amenazas internas.
    - Interrupción de servicio.
    - Mala configuración del entorno. Errores en la configuración del sistema, servidores o contenedores que pueden dejar expuestos servicios, datos o funcionalidades críticas.
    - Daño reputacional. Puede conllevar un impacto negativo sobre una empresa debido a fallos de seguridad, pérdida de datos o mal funcionamiento de la aplicación.
    - Vulnerabilidades en dependencias, pueden ser explotadas si no se actualizan o revisan.
    - Amenazas internas, podrían implicar empleados que abusan de su acceso para robar o manipular datos.

- **Diseñar la aplicación web de forma segura:**

  Lo primero que haría sería separar la aplicación en tres componentes principales: frontend, backend y base de datos. A continuación, definiría claramente el flujo de datos y los puntos de acceso, exponiendo únicamente los servicios necesarios. De esta forma, se pueden identificar posibles amenazas desde la fase de diseño y prevenir riesgos desde el inicio del desarrollo. Además, utilizaría Docker para aislar los distintos servicios, ejecutándolos en contenedores independientes, lo que mejora la seguridad y el control del entorno. Por último, aplicaría un enfoque de seguridad multicapa, es decir, implementar varias capas de protección en el sistema. De este modo, si una de las capas falla, las demás continúan protegiendo la aplicación.

- **Qué controles pondrías sobre acceso, datos, red y software:**

  Introduciría un sistema de detección de intrusiones (IDS) para monitorizar la red o los sistemas, detectar actividades maliciosas y generar alertas. Luego, introduciría un control de acceso a los datos. De esta forma, podemos proteger la información tanto en tránsito como en reposo, cifrar la comunicación y realizar copias de seguridad. A nivel de red, intentaría exponer los servicios mínimos necesarios y utilizar protocolos seguros como TLS. Por último, en el ámbito del software, intentaría implementar y programar de forma segura, realizando análisis SAST y DAST, así como la revisión de dependencias. Además, mantendría el sistema actualizado mediante parches y actualizaciones.

- **Qué principios de Zero Trust aplicarías:**

  Entonces, para implementar Zero Trust, cada usuario y dispositivo de conexión debe ser verificado, aunque esté dentro de la red. Esto implica verificación continúa, validando siempre el acceso. Por ejemplo, mediante multifactor (MFA), identidad, contexto, etc. También limitaría el radio de explosión, reduciendo el impacto de un ataque. Para ello, aplicaría mínimo privilegio y microsegmentación, dividiendo la red en pequeñas partes para controlar y limitar el acceso entre ellas. Además, incorporaría automatización y contexto, recopilando datos y respondiendo automáticamente a amenazas. Esto se puede implementar mediante visibilidad, conociendo usuarios, redes y dispositivos, y centralizando la información mediante un SIEM, un sistema que recopila, procesa y genera alertas sobre la información de ciberseguridad de la organización. De esta manera, cuando ocurre un fallo, se puede mitigar mediante una respuesta rápida, reduciendo el impacto del ataque. Esto mejora los procesos, haciéndolos más eficientes y seguros, sin necesidad de un gran aumento de recursos.

4. Desarrollo seguro (S-SDLC)

La seguridad debe integrarse desde el inicio en todas las fases del ciclo de vida. Esto quiere decir que desde la primera fase de recopilación de requisitos ya deberíamos establecer parámetros de seguridad. En caso de que la seguridad se deje al final, puede tener varias consecuencias: no se detectan fricciones, los problemas aparecen más tarde, y esto puede ser un gran problema porque aplicar medidas de mitigación será más costoso y complejo de solucionar.

Fases del S-SDLC:

1. **Recopilación de requisitos**
    - Evaluación de amenazas: identificar y analizar posibles amenazas al sistema con el objetivo de que nuestra aplicación web sea estable y segura. En este caso, al desarrollar una aplicación web, las amenazas potenciales podrían incluir ataques de inyección SQL, ataques de XSS, ataques de DDoS, etc.
    - Evaluación de riesgos: ayuda a entender el impacto de estas amenazas y la probabilidad de que ocurran. El riesgo asociado con un ataque DDoS podría ser una interrupción de servicio que conduce a la insatisfacción del cliente y, posiblemente, genere pérdidas financieras y daño a la reputación.
2. **Diseño seguro**
    - Mapeo de ataques: identificar posibles amenazas en nuestro sistema y determinar medidas para mitigarla.
        a) Modelado del sistema: representar la arquitectura, componentes y flujos de datos.
        b) Identificar amenazas: se trata de identificar posibles amenazas a cada parte del sistema.
        c) Evaluación de riesgos.
        d) Estrategias de mitigación para reducir la probabilidad, es decir, si la probabilidad de que ocurra un ataque DDoS en nuestra aplicación web es del 80%, vamos a intentar reducir ese número al 20% o incluso menos.
        e) Validación continua para comprobar que las medidas funcionan.
3. **Implementación**
    - Pruebas estáticas, SAST (probar el código antes de entrar en producción).
        a) Elegir la herramienta, esto sobre todo depende de qué lenguaje de programación se haya utilizado, el presupuesto y los requisitos específicos del proyecto.

b) Integrar SAST en el proceso de desarrollo: esto nos permitirá a los desarrolladores detectar y solucionar problemas de seguridad mientras programan.
c) Definir políticas de seguridad: definir qué tipos de problemas debe buscar la herramienta en nuestra aplicación web, ya que no se puede entrar a producción con vulnerabilidades críticas.
d) Analizar los resultados: una vez ejecutada la herramienta SAST, se producirá una lista de posibles problemas de seguridad.
e) Remediar vulnerabilidades: evaluar los problemas de seguridad, que deben solucionarse (cambiando partes de nuestro código, actualizando bibliotecas, modificando configuraciones, etc).

- Verificación de las dependencias, SCA: las herramientas SCA se utilizan para automatizar el proceso de identificación de componentes de open-source en una base de código.
    a) Verificación contra bases de datos de vulnerabilidades: qué librerías y subcomponentes han sido usados para crear la aplicación web.
    b) Generar una lista de materiales: qué software hay por debajo implementado para conseguir que todo funcione (Ejemplo: tenemos una librería de Python que es vulnerable, pero mi código no lo es).
    c) Analizar y remediar: una vez obtenido el listado podemos ver cuáles son vulnerables y proponer la versión que hay que parchear.

4. Pruebas

- Análisis de artefactos (código fuente): consiste en revisar los elementos generados tras la construcción de la aplicación, como el código compilado, archivos de configuración y la imagen Docker.
    a) Identificar el código que será analizado (la imagen del contenedor o los archivos backend/frontend).
    b) Escoger una herramienta para analizar posibles vulnerabilidades o configuraciones inseguras.
    c) Revisar resultados, aquí es donde podríamos clasificar las vulnerabilidades por severidad e ir intentando corregir los problemas detectados en orden descendente (mayor severidad == mayor daño, por lo tanto, vamos a empezar con las vulnerabilidades más críticas dentro de nuestra aplicación web).
    d) Documentar las acciones realizadas.

- Análisis dinámico (DAST): se utiliza cuando la aplicación está en ejecución y permite detectar vulnerabilidades desde el exterior, simulando ataques reales (XSS, SQL injection, etc).

5. **Preparación y despliegue**
    - Monitoreo de degradación (calidad de seguridad): para detectar ataques, fugas y nuevas vulnerabilidades con el tiempo. Por ejemplo, si se observa en nuestra aplicación web que el tiempo de respuesta aumenta de forma anormal o el consumo de los recursos ha aumentado de forma drástica, esto podría indicar un ataque de tipo DoS o una fuga de memoria.
    - Prueba de regresión de seguridad: se trata de un conjunto de pruebas de validación constantes para identificar posibles amenazas. Un ejemplo claro de esto podría ser que ya habíamos parcheado una vulnerabilidad de inyección SQL, pero luego, al actualizar el código, comprobamos que ese parche ya no es efectivo y vuelve a aparecer.Esto suele ser bastante común no solo dentro de nuestra aplicación web, sino también en nuestro sistema operativo Windows, que a veces tenemos una versión que no tiene “x” vulnerabilidad y, al realizar la actualización de nuestro equipo, esa vulnerabilidad pasa a existir.
6. **Mantenimiento y monitoreo**
    - Monitoreo adversario: consiste en recopilar y analizar información (incluyendo la dark web) para anticipar ataques reales y planificar medidas preventivas.
    - Mantenimiento como prevención de riesgos: aplicar parches, realizar auditorías periódicas, monitorizar continuamente para que todo funcione correctamente.
        a) Formar a los usuarios (prevenir riesgos), planes de recuperación antes desastres y cumplimiento de las normas de seguridad. Además estas medidas se alinean con estándares de ciberseguridad como NIST.